

비행 시뮬레이터 개발 입문

flight-sim2 프로젝트로 배우는 항공 시뮬레이션 분야의 처음부터 끝까지

함께 공부하는 가이드

2026-04-28

0. 들어가며 — 이 문서를 읽는 법

이 문서는 우리가 함께 만든 `flight-sim2` 프로젝트를 출발점으로 삼아, **비행 시뮬레이션이라는 산업 분야 전체의 지도**를 그려보는 입문 가이드다. 분야를 처음 접하는 사람이 첫 페이지부터 마지막 페이지까지 차례대로 읽으면, 다음을 알게 된다.

- 비행 시뮬레이터가 정확히 **무엇**이며 왜 만드는가
- 우리가 만든 코드의 각 부분이 **어떤 학문적·산업적 배경**에서 나왔는가
- 전 세계 어떤 **회사들**이 이 분야에서 일하며, 어떤 방식으로 개발하는가
- 한국**에는 어떤 플레이어가 있고 어떤 기회가 있는가
- 우리 프로젝트를 **산업급**으로 키우려면 무엇을 더 해야 하는가
- 더 깊게 공부하고 싶을 때 어디로 가야 하는가

핵심 원칙: 모든 추상 개념은 **우리 프로젝트의 실제 코드 줄과 매핑**해서 설명한다. “이 기능이 왜 거기 있는지” 모르겠으면, 해당 챕터의 마지막 단락을 보면 코드 위치가 적혀 있다.

1. 비행 시뮬레이션이란? — 분야 전체 지도

1-1. 한 줄 정의

비행 시뮬레이션 = 실제로 항공기를 띄우지 않고도 그 거동을 컴퓨터로 재현하는 일.

이 한 줄을 펼치면 네 가지 하위 시장이 나온다.

분류	목적	대표 제품/회사	시장 규모 (2024)
엔터테인먼트	즐거움, 게임	Microsoft Flight Simulator, X-Plane, DCS World	연간 ~\$2B
조종사 훈련	면허/기종 전환 훈련	CAE, FlightSafety, L3Harris	연간 ~\$8B
방산	전투 시나리오, 무기 통합	Lockheed Martin, BAE, Leonardo, KAI	연간 ~\$15B
엔지니어링/연구	항공기 설계 검증, 자율비행 알고리즘 개발	Boeing/Airbus 사내, NASA, JSBSim	비공개 (대형)

각 시장은 요구되는 정밀도, 인증, 가격, 개발 사이클이 완전히 다르다.

- 게임은 30 ms 안에 그림이 나오면 OK
- 훈련 시뮬은 FAA Level D (실제 항공기와 구분 불가능 수준) 인증을 받아야 한다
- 방산은 거기에 더해 보안 인증과 무기 시스템 통합까지

1-2. 우리 프로젝트는 어디?

flight-sim2 는 명시적으로 엔지니어링/연구의 가벼운 버전 + 엔터테인먼트 데모를 노린다. PRD §1에 적힌 “LabVIEW/FPGA 기반 시뮬레이션의 웹 시각화 SaaS 프로토타입”이라는 문장이 핵심.

쉽게 말해: - 우리는 진짜 보잉 737 시뮬레이터를 만드는 게 아니라 - 어떤 회사가 자기네 비행 알고리즘을 만들었을 때, 그걸 웹 브라우저에서 즉시 시각화해주는 도구를 만든다 - 하드웨어를 사거나 Unity 라이선스를 살 필요 없이, 단일 HTML 파일을 보내주면 누구든 바로 볼 수 있다

→ 프로젝트의 HITL 모드(H 키)가 정확히 이 시나리오다. 외부 시스템이 비행 상태를 보내면 우리는 그저 보여준다.

2. 왜 Three.js? — Unity/Unreal을 안 쓴 이유

2-1. 게임 엔진의 강점과 약점

기준	Unity	Unreal	Three.js
그래픽 품질 (out-of-box)	★★★★	★★★★★	★★
학습 곡선	중간	가파름	완만
라이선스 비용	매출 구간별 royalty	5% royalty	MIT (무료)
배포 마찰	빌드 + 다운로드	빌드 + 다운로드	URL 한 줄
즉시성	빌드 5-30분	빌드 10분-2시간	새로고침 0.5초
물리 직접 제어	어려움 (PhysX 추상)	어려움 (Chaos 추상)	쉬움 (직접 작성)

우리가 Three.js를 고른 이유 3가지:

1. 물리를 우리가 직접 통제해야 했다. 비행은 일반 강체 충돌이 아니라 **공력**이 본질이다. PRD §6 “거부” 항목에 Cannon.js / Ammo.js (Unity의 PhysX와 같은 강체 엔진) 를 명시적으로 거부한 이유다.
2. **즉시성**. 사용자(데모 대상)가 “보여줘” 라고 하면 URL 하나 보내면 끝나야 한다. Unity/Unreal 빌드를 메일로 보내고 압축 풀고 실행시키게 하면 데모 가치가 절반으로 떨어진다.
3. **단일 HTML**. 단일 파일이라는 제약이 디자인 전체에 일관성을 강제한다. 외부 의존성 0개, 빌드 도구 0개. 우리 프로젝트가 5000 LOC를 넘었어도 여전히 `npm run bridge` 한 줄이면 동작한다.

→ 코드: `index.html` (CDN으로 Three.js r128 로드), `src/main.js` (부트스트랩)

2-2. 그러면 게임 엔진은 누가 쓰나?

- **Microsoft Flight Simulator 2024** (Asobo Studio): 자체 엔진 (Unity/Unreal 아님)
- **DCS World** (Eagle Dynamics): 자체 엔진
- **Lockheed Martin Prepar3D**: Microsoft ESP 엔진 fork
- **CAE 훈련 시뮬**: 자체 엔진 (Medallion-6000 시리즈)
- **AirSim** (Microsoft Research, 단종): **Unreal Engine** 위
- **JoyCity, KRAFTON 같은 한국 게임사**: Unity/Unreal

산업급으로 갈수록 자체 엔진을 쓰는 경향이 있다. 이유는 **렌더링 vs 시뮬레이션의 비율**이 엔진과 다르기 때문. 조종사 훈련 시뮬은 GPU의 70%를 콕핏 디스플레이 정확도에, 30%만 외부 화면에 쓴다 — 게임 엔진의 가정과 정반대.

3. 비행 물리의 ABC — 코드와 함께

이 챕터는 우리 `src/physics.js` 를 한 줄씩 따라가며 항공 역학의 기초를 배운다.

3-1. 네 가지 힘

비행기에 작용하는 힘은 항상 네 가지다.



비행기가 떠 있다 = 양력 $\geq$ 중력. 비행기가 가속한다 = 추력 $\geq$ 항력.

양력 공식

$$L = \frac{1}{2} \cdot \rho \cdot v^2 \cdot S \cdot CL$$

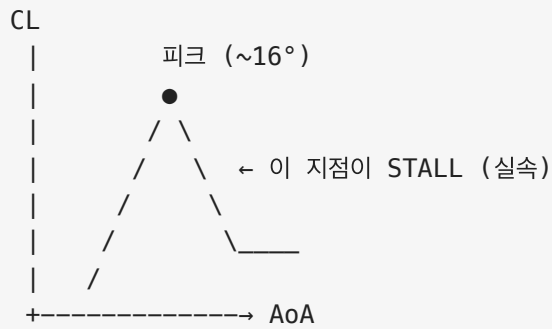
- ρ (rho): 공기 밀도 (해수면에서 1.225 kg/m^3)
- v : 대기 속도 (m/s)
- S : 날개 면적 (m^2)
- CL : 양력 계수 (받음각의 함수)

이 공식은 베르누이의 정리와 나비에-스토크스 방정식을 단순화한 것으로, 1900년대 초 NACA(NASA의 전신)가 정립했다. 라이트 형제가 1903년 첫 비행을 성공한 직후 NACA가 1915년에 설립되면서 미국이 항공 역학의 표준을 만들기 시작한다.

→ 코드: `src/physics.js` `liftForce()`, `dragForce()`. 단위 테스트: `tests/physics.test.mjs`.

받음각 (Angle of Attack, AoA)

날개가 공기와 만나는 각도. AoA가 커지면 양력 계수(C_L)가 비례해서 커진다 — 단, 어느 임계점까지만.



피크를 넘으면 공기가 날개 위에서 분리되어 양력이 갑자기 줄어든다. 이게 **실속(stall)**. 비행기 사고의 큰 원인이며, 모든 시뮬레이션의 핵심 모델이다.

→ 코드: `src/physics.js` `liftCoefficient()`. STALL 상태가 되면 HUD가 빨간색으로 바뀌고, 우리는 사운드 (`src/audio.js`)에서 880Hz 4Hz pulse 비프 경고음까지 낸다.

3-2. 6-DOF (Six Degrees of Freedom)

항공기는 3차원 공간에서 **6개의 자유도**로 움직인다.

	Translation (위치)	Rotation (자세)
X 축	Surge (앞뒤)	Roll (롤, 좌우 기울기)
Y 축	Heave (위아래)	Pitch (피치, 코 위아래)
Z 축	Sway (좌우)	Yaw (요, 좌우 회전)

조종사가 다루는 것은 회전 3개 + 추력 1개 = 4개 입력으로, 6-DOF를 제어한다 (병진은 회전과 추력의 결과). 우리 코드:

- W/S = pitch
- A/D = roll
- Q/E = yaw
- ↑/↓ = throttle (추력)

→ 코드: `src/main.js` `stepPhysics()` 함수가 매 프레임 6-DOF를 적분한다.

3-3. 좌표계 — 가장 헷갈리는 부분

산업 표준 좌표계는 두 개를 동시에 쓴다.

Body Frame (기체 프레임)

- 항공기 무게중심 기준
- +X = 우측 날개
- +Y = 동체 위

- $-Z$ = 기수 (forward)

이 컨벤션은 **MIL-STD-1797A** (미군 항공기 비행 품질 군사 표준) 와 **AIAA 표준**에서 가져왔다.

NED (North-East-Down)

- 지표면 기준
- N = 북쪽 (+X)
- E = 동쪽 (+Y)
- D = 아래 (+Z, 중력 방향!)

NED에서 Down이 양수인 게 직관에 어긋나지만, **중력 가속도가 양수로 표현되어 적분이 깔끔해서** 항공/항법 분야 표준이다.

Three.js 좌표계

- $+Y$ = up (NED와 부호 반대!)
- $-Z$ = forward (Body의 $-Z$ 와 일치)

이 부호 차이 때문에 우리 `bridge/server.mjs` 의 `relayTelemetry()` 에서 시뮬 좌표를 NED로 변환하는 코드가 있다:

```
const vN = Math.round((-t.vz) * 100); // sim -Z = North
const vE = Math.round(( t.vx) * 100); // sim +X = East
const vD = Math.round((-t.vy) * 100); // sim -Y = Down
```

이 한 함수에서 부호를 잘못 쓰면 QGroundControl 지도에서 비행기가 정반대로 날아간다. 실제 산업체에서도 좌표계 부호 버그는 가장 자주 일어나고 가장 디버깅이 어려운 종류다.

→ 코드: 모든 모듈 상단에 `// COORDINATE: ...` 주석을 박아둔 이유. CLAUDE.md §3 참조.

4. 실세계 회사들은 어떻게 만드나

📄 두 개의 심층 자료: - [INDUSTRY-REPORT.md \(PDF\)](#) — 광역 산업 (도담/KAI/한화/슈퍼널/CAE 등) - [PEERS-LANDSCAPE.md \(PDF\)](#) — 우리 프로젝트와 같은 기술 모델만 (GeoFS, Helios GCS, Auterion Virtual Skynode, 파이온 OMNI-GCS 등)

아래는 광역 요약. 우리 기술 모델의 직속 동료만 보고 싶으면 PEERS-LANDSCAPE 로.

4-1. 글로벌 게임 시뮬 시장

Microsoft Flight Simulator 2020 / 2024

- 개발사: **Asobo Studio** (프랑스 보르도, 직원 ~400명)
- 퍼블리셔: Microsoft / Xbox Game Studios
- 엔진: 자체 (DirectX 12)
- 지형: Bing Maps + Azure AI 데이터 → 전 세계 1:1 스케일
- 특징: 실시간 기상 (METAR), 1:1 도시 모델, 200+ 항공기

이 회사는 1999년 자동차 게임으로 시작했지만 2017년 Microsoft가 새 Flight Simulator를 만들 파트너로 발탁. **Azure 클라우드와 결합**해서 게임 영역에 클라우드 시뮬을 처음 정착시켰다.

X-Plane 12

- 개발사: **Laminar Research** (미국 콜로라도, 직원 ~30명)
- 창업자: Austin Meyer (혼자 1992년부터)
- 특징: **Blade Element Theory** 기반 정확한 공력. 가상 풍동을 실시간으로 시뮬레이션
- FAA Level D 인증된 유일한 게임 출신 엔진

X-Plane은 **공력 모델 자체로 인증을 받은** 거의 유일한 사례. 진짜 비행 훈련에 쓸 수 있다는 뜻.

DCS World

- 개발사: **Eagle Dynamics** (스위스 본사, 러시아·우크라이나 개발팀)
- 특징: 군용기 정밀 시뮬 ("study sim"). F-16, F-18, A-10 등이 실제 매뉴얼 수준으로 작동
- 라이선스: 기본 무료 + 항공기별 DLC (\$60-80)

DCS는 게임으로 위장한 방산 시뮬에 가깝다. 실제로 미 공군이 일부 훈련에 쓴 적도 있다.

4-2. 훈련 / 방산 시뮬 시장

CAE (Civil Aviation Electronics) — 캐나다

- 1947년 설립, 시가총액 ~\$8B
- 전 세계 민간 조종사 훈련 시장 점유율 **70%+**
- 보잉/에어버스 항공기마다 풀 모션 시뮬을 만든다
- 자체 엔진: **Medallion-6000** (실시간 영상), **Tropos** (지형)

조종사가 보잉 737 자격증을 따려면 거의 100% CAE 시뮬을 거친다.

L3Harris Technologies — 미국

- 합병체 (L3 + Harris, 2019)
- 항공 + 우주 + 해양 시뮬
- 미군 F-16, F-22 시뮬 다수 납품

Lockheed Martin Prepar3D (“P3D”)

- Microsoft FSX의 군용 fork
- 학교/일부 군 사용. Microsoft가 게임용으로 다시 가져온 후 점유 줄어들

BAE Systems / Leonardo

- 영국 BAE: Eurofighter Typhoon 시뮬
- 이탈리아 Leonardo: M-346, F-35 합작

FlightSafety International

- 1951년 설립, Berkshire Hathaway (워런 버핏) 자회사
- CAE의 미국 라이벌
- Gulfstream 등 비즈니스 제트 훈련에 강함

4-3. 한국 — 우리 컨텍스트

KAI (한국항공우주산업)

- 1999년 설립, 사천 본사
- T-50 골든이글, FA-50 (말레이시아·폴란드 수출 성공), KF-21 보라매
- **자체 비행 시뮬레이터 (TRD, Training Research Device) 개발**
- 록히드마틴과 합작 (T-50은 F-16 디자인 라이선스)

LIG Nex1

- 2014년 LG넥스원에서 분리
- 미사일·레이더·통신·EW (전자전)

- **드론 + UAM 시물**: 군용 무인기 훈련 시물 시장 1위 국내
- **본사**: 성남 (판교)

한화시스템

- 한화 그룹 방산 IT 부문
- 통합 시뮬레이터 (지상·항공 통합)
- KF-21 미션 컴퓨터 일부 담당

두산모빌리티이노베이션

- 두산 그룹 드론 자회사
- 수소연료전지 드론 (DS30W)
- **PX4 기반** 비행 제어 사용 → 우리가 쓴 MAVLink 같은 표준 사용

베셀에어로스페이스

- 사천 위치, KAI 협력사
- 시뮬레이터 통합 + 기술 인력 파견

신생/UAM 스타트업

- **베타테크놀로지스 코리아** (미국 베타)
- **현대차 슈퍼널** (UAM eVTOL, 2028 양산 목표)
- **Plana** (한국 UAM 스타트업, 하이브리드 eVTOL)
- **두시텍, 유콘시스템** (소형 무인기)

→ 한국 시장은 **방산 (KAI/LIG/한화) + UAM (현대 슈퍼널, Plana) + 드론 (두산/유콘) 세 갈래**. 우리 프로젝트의 SaaS 각도는 후자 두 그룹에 호소력이 크다.

4-4. UAM / 드론 신시장 (글로벌)

이 시장이 가장 빠르게 자라는 곳이고, 우리 프로젝트가 정확히 겨냥하는 영역.

eVTOL (전기 수직이착륙기)

- **Joby Aviation** (미국, 시총 ~\$5B): Toyota 투자, FAA 인증 진행 중
- **Archer Aviation**: United Airlines/Stellantis 투자
- **Lilium** (독일, 2024 파산 후 재구조)
- **Volocopter** (독일, 2024 파산 후 재구조)
- **Eve Air Mobility** (브라질 Embraer 자회사)

드론 자율비행

- **Skydio** (미국): 자율비행 드론 1위, 군용도 진출
- **Anduril**: 방산 + 자율 무기. AI 통합
- **Shield AI**: 드론 경찰 AI

시뮬 부문

- **Microsoft AirSim** (2017-2022): 자율비행 시뮬, **Unreal Engine 기반**, 단종
- 후속: **Cosys-Lab AirSim Fork**, **ProjectAirSim** (Microsoft 새 프로젝트)
- **NVIDIA Isaac Sim**: 로보틱스/드론, **Omniverse 기반**
- **Open Source**: **Gazebo** (ROS 표준), **PX4 SITL**, **jMAVSim**

→ 우리 프로젝트의 brige + HITL 모델은 정확히 PX4 SITL의 웹 버전이라 볼 수 있다.

5. 오픈소스 생태계 — 우리가 디딘 어깨

5-1. PX4 / ArduPilot — 드론 자동조종

PX4

- **PX4 Foundation** (Linux Foundation 산하) 운영
- 2009년 ETH 취리히에서 시작
- 현재 50+ 회사가 코어 기여 (Skydio, Auterion, Sony 등)
- 라이선스: **BSD-3** (상업 사용 가능)
- 특징: 모듈러, NuttX RTOS, 임베디드 + Linux 양쪽

ArduPilot

- 2010년 시작 (Chris Anderson, 전 Wired 편집장)
- 더 큰 커뮤니티, 호환 하드웨어 다양
- 라이선스: **GPL-3** (상업 사용 시 소스 공개 의무)

PX4는 상업 친화적, ArduPilot은 커뮤니티 친화적. 둘 다 **MAVLink**로 통신 — 그래서 우리가 만든 브릿지가 양쪽과 다 통한다.

5-2. QGroundControl — 우리가 통합한 GCS

- **MAVLink Foundation + Auterion** (스위스) 주도 개발
- 2014년부터, 처음엔 PX4 도구로 시작 → 범용으로 진화
- **Qt 6 + C++**, 모든 OS
- 라이선스: **Apache 2.0**

GCS = Ground Control Station. 드론을 지상에서 조작하는 소프트웨어. 우리 코드 (`bridge/server.mjs` + `bridge/mavlink.mjs`) 가 QGC를 만족시키느라 한 모든 것은 PX4/ArduPilot 진영의 사실상 표준을 따르는 일이었다.

5-3. ROS + Gazebo — 로봇틱스 시뮬 표준

- **ROS** (Robot Operating System): Open Source Robotics Foundation, 2007년 시작
- **Gazebo**: 3D 물리 시뮬레이터, ROS와 짝
- 학계 + 기업 연구실의 사실상 표준

자율주행차 + 드론 + 로봇팔 다 같은 스택을 쓴다. 우리 프로젝트는 일부러 ROS를 안 썼는데 (웹 우선), 만약 산업체로 발전 시킨다면 ROS bridge 추가가 자연스러운 다음 단계.

5-4. JSBSim — 정통 공력 모델

- 1997년 시작, NASA + 미군 일부 사용
- **6-DOF, 실제 NACA 데이터 기반**
- C++ 라이브러리, FlightGear의 코어 엔진
- 모델 파일 (XML)에 항공기 공력 계수 정의 → 진짜 항공기 거동 재현

우리 `physics.js` 는 JSBSim의 단순화 버전. **산업급으로 가려면 JSBSim 통합이 큰 다음 단계** (M11쯤).

5-5. FlightGear

- 1997년 시작, 가장 오래된 오픈소스 비행 시뮬
- JSBSim + 자체 렌더러
- 학교/연구용으로 여전히 활발

→ 즉 우리 코드 (`src/physics.js`)에서 사용한 양력/항력 공식은 **NACA → JSBSim → 우리 단순화**의 계보를 거쳤다.

6. MAVLink — 우리가 쓴 표준의 정체

6-1. 무엇인가

- 드론 ↔ 지상국 통신 프로토콜
- 2009년 ETH 취리히 (PX4와 같이 시작)
- 이진 프레임 (XML 정의 → 자동 코드 생성)
- 100+ 메시지 타입: HEARTBEAT, ATTITUDE, GLOBAL_POSITION_INT, VFR_HUD, MISSION_ITEM_INT 등

6-2. 누가 만들고 유지하나

- **MAVLink Foundation** (Dronecode Foundation 산하)
- 메인테이너: Auterion, ArduPilot 팀, PX4 코어 기여자들
- 표준 정의: <https://mavlink.io/>

6-3. 우리가 직접 인코더를 만든 이유

`bridge/mavlink.mjs` 한 파일에 v1 인코더 + 디코더를 직접 썼다. 다른 옵션이 있었음에도 그렇게 한 이유:

- **node-mavlink** npm 패키지: 무겁고 의존성 많음
- **mavlink-browser**: 메인테넌스 안 됨
- 우리에게 필요한 메시지: 10개 정도

PRD §6 “외부 의존성 최소화” 원칙이 직접 구현을 선택하게 만든 케이스. **CRC-16/MCRF4XX**, **field reorder** (필드 크기 내림차순 정렬), **CRC_EXTRA byte** (메시지마다 다른 상수) — 이 세 가지가 MAVLink의 wire format 핵심.

→ 코드: `bridge/mavlink.mjs` `crc16()`, `encodePacket()`. 표준 검사값 `0x6F91` for `"123456789"` 으로 단위 테스트.

6-4. 왜 표준이 되었나 (그리고 DJI는 안 따른다)

MAVLink가 사실상 표준이 된 이유: 1. PX4와 ArduPilot이 둘 다 채택 2. QGroundControl + Mission Planner 둘 다 지원 3. 오픈소스라 누구든 쓸 수 있음

하지만 **DJI**는 자체 SDK (Mobile SDK + Onboard SDK) 사용. 시장 1위라 그래도 됨. **Skydio**도 자체. 군용은 종종 자체 보안 프로토콜.

→ 우리 코드는 PX4/ArduPilot/QGC 호환. DJI 호환은 별도 작업 필요 (각자의 SDK).

7. 우리 코드의 학문적 출처 매핑

이 표는 우리 프로젝트의 각 모듈이 어떤 학문적·산업적 전통에서 왔는지 보여준다.

우리 코드	개념	출처 / 표준	시기
physics.js 양력 공식	베르누이 + NACA airfoil	NACA (1915-)	1900s
physics.js 실속 모델	C_L - α 곡선	Anderson “Aircraft Performance” 교과서	-
physics.js 공기 밀도	표준 대기 모델 (US Standard Atmosphere 1976)	NASA	1976
autopilot.js cascade	Outer/inner loop PID	Honeywell 자동조종 표준	1950s+
collision.js sphere-vs-box	GJK / SAT 단순화	게임 물리 (Christer Ericson 책)	-
damage.js HP 모델	군용 대미지 모델 (“kill probability”)	DIS 표준	1990s
effects.js 파티클	Reeves 파티클 시스템	1983 (Pixar Lucasfilm 시절)	1983
recorder.js 링버퍼	Flight Data Recorder (블랙박스)	ICAO Annex 6	1958+
hitl.js HITL 패러다임	Hardware-in-the-loop	DARPA, 항공 R&D	1990s
bridge/mavlink.mjs	MAVLink v1 wire format	Dronecode (2009-)	2009
gamepad.js 데드존	게임 컨트롤러 표준 처리	Microsoft XInput	2005
audio.js 절차적 오디오	Web Audio API + 합성	W3C 표준	2011+
world.js fbm noise	Perlin/Worley noise	Ken Perlin (1985)	1985
world.js 스카이 셰이더	Rayleigh + Mie scattering	단순화	-
multiplayer.js SSE relay	게임 netcode (delta + interpolation)	Quake 3 architecture	1999
WebXR	W3C WebXR Device API	W3C	2018+

이 표를 한 번 천천히 읽으면, 우리 5000줄 코드가 사실은 **한 세기에 걸친 항공/CG/네트워크 표준의 모자이크**라는 게 보인다.

8. 실무에서의 개발 방식 (회사별)

8-1. 게임 스튜디오 방식 (Asobo, Eagle Dynamics)

- **방법론:** 스크럼 + 2주 스프린트
- **언어:** C++ (코어), C# (도구), Python (파이프라인)
- **품질:** 비주얼/게임플레이 우선, 인증 X
- **사이클:** 2-4년 (한 타이틀)
- **팀:** 50-400명

8-2. 방산 / 훈련 시뮬 방식 (CAE, L3Harris, KAI)

- **방법론:** V-Model (Validation & Verification 강제)
- **표준:** DO-178C (FAA 항공 SW 인증, level A-E)
- **언어:** C / C++ / Ada (구식 시뮬은 Ada95 여전)
- **품질:** 100% 코드 커버리지, 형식 명세
- **사이클:** 5-10년 (한 시뮬레이터)
- **팀:** 100-1000명

DO-178C는 FAA가 항공기 SW에 요구하는 인증 규정. 5단계 (Level A: 사람이 죽을 수 있음 ~ Level E: 영향 없음). 우리 코드는 DO-178C 어떤 레벨도 만족 안 함 (현재는 그게 목표가 아님).

8-3. UAM/드론 스타트업 방식 (Skydio, Joby, Anduril, 슈퍼널)

- **방법론:** Agile + DevOps + 오픈소스 기여
- **언어:** C++, Rust, Python, **TypeScript** (시각화)
- **품질:** 시뮬레이션 in-the-loop test (수백만 시간)
- **사이클:** 6개월 ~ 2년
- **팀:** 10-100명 (집중도 높음)

→ 이 그룹이 **우리 프로젝트의 잠재 고객/파트너**. 이들은 PX4/ArduPilot/Gazebo 같은 OSS를 적극 사용 + 자체 도구 개발.

8-4. 한국 방산 (KAI/LIG/한화) 방식

- **혼합:** 미국 방산과 게임 스튜디오의 중간
- **언어:** C/C++ 메인, MATLAB/Simulink 모델링, **LabVIEW** (테스트 장비)
- **품질:** KAS-GW 인증 (한국 방산 SW 인증)
- **사이클:** 10년+ (KF-21처럼)

→ PRD §1의 “LabVIEW/FPGA 시뮬 SaaS” 각도가 이 그룹과 정확히 맞물린다. MATLAB/Simulink 모델을 LabVIEW로 통합한 후 우리 시뮬에 HITL로 보내면 그게 데모.

9. 표준 / 인증 — “진짜” 시뮬이 통과하는 것들

9-1. FAA Level A/B/C/D — 조종사 훈련 시뮬

- **Level A:** 정적 비행 모델, 조작 인터페이스만. 절차 훈련용.
- **Level B:** 동적 모델 + 사운드. 시각 시스템 가능.
- **Level C:** 풀 모션 + 전 시계 시각. 야간 + 일부 일중 조건.
- **Level D:** 모든 비행 조건 재현 가능. 실제 항공기와 구분 불가능. 0 시간 비행으로 자격증 가능.

CAE의 Level D 시뮬 한 대 가격: **\$10-15M USD**. 조종사 한 시간 훈련 비용: \$500-1000.

우리 프로젝트는 Level A조차 안 됨 (정확도 부족). 게임 시뮬에 가깝다.

9-2. DO-178C — 항공 SW 인증

- 미 RTCA 발행, FAA/EASA 채택
- A (catastrophic) ~ E (no effect) 5단계
- 100+ objectives, 각 단계마다 더 엄격한 증명 필요
- **요구:** 요구사항 → 설계 → 코드 → 테스트의 100% 추적성

우리 코드는 unit test로 단위 함수 일부만 검증. 산업급은 **모델 기반 개발 (Simulink) → 자동 코드 생성 → MC/DC 100% 커버리지**.

9-3. DIS / HLA — 분산 시뮬

- **DIS** (Distributed Interactive Simulation, IEEE 1278): 1990년대 미군 표준
- **HLA** (High Level Architecture, IEEE 1516): DIS의 후속, 더 일반화
- 여러 시뮬레이터를 네트워크로 연결 (예: 항공 + 지상 + 해양 시나리오)

우리 멀티플레이어 (`src/multiplayer.js`) 는 매우 단순화된 DIS의 정신을 따른다. 산업 시뮬은 HLA로 NATO 합동 훈련까지 한다.

9-4. ARINC 661 — 콕핏 디스플레이 표준

- 글래스 콕핏 디스플레이 인터페이스 표준 (Airbus 주도)
- XML 명세 → 인증 가능한 디스플레이 코드
- A380, A350, A220 등 사용

우리 PFD (`src/hud.js` 의 heading rose, tape) 는 ARINC 661 스타일을 흉내냈지만 표준 준수는 아님.

10. 우리 프로젝트를 산업급으로 키우는 길

현재 위치 → 산업급까지의 마일스톤 추천:

10-1. 단기 (1-3개월, 1인 작업)

- JSBSim 통합: WASM 빌드 → 우리 `physics.js` 를 정통 공력 모델로 교체 → 인증급 재현
- 실제 지형: Mapbox terrain tiles 또는 SRTM 데이터 → 인천공항 주변 진짜 모습
- 사운드 디테일: 실제 항공기 녹음 샘플 (MSFS 같은 게임의 ambient sample 라이선스 가능)
- AP 정확도 완성: PID + integral wind-up protection + 실 항공기 거동 fidelity

10-2. 중기 (3-12개월, 2-5인 팀)

- 다중 항공기 클래스: 헬리콥터 (cyclic + collective + tail rotor), VTOL, 전투기
- 콕핏 인터리어: 폴 글래스 콕핏 + ARINC 661 호환 인스트루먼트
- 시나리오 엔진: 미션 에디터, 절차 훈련 모드
- HLA bridge: 군용 분산 시뮬과 연동
- 사운드 엔진: FMOD 또는 Wwise 통합 (라이선스)

10-3. 장기 (1-3년, 정식 SaaS)

- DO-178C Level D 인증 path: 형식 명세 + 100% 커버리지 + 외부 감사
- 하드웨어 통합: 폴 모션 플랫폼 (3DOF/6DOF 헥사포드) 인터페이스
- 다중 사용자 cloud SaaS: 동시 1000+ vehicle, 클라우드 GPU 렌더링
- B2B 라이선스: 방산/UAM 회사에 SDK + integration 서비스 판매

10-4. 사업적 각도 (PRD §1 각도)

`flight-sim2` 의 SaaS 잠재력을 진지하게 평가하면:

- 표적 고객: 한국 UAM 스타트업 (슈퍼널, Plana), 드론 회사 (두산, 유콘), 방산 R&D 부서 (LIG/KAI 산하 연구소)
- 차별화: 단일 HTML, 즉시 데모, 외부 sim ↔ 웹 시각화의 어댑터 역할
- 수익 모델: 개인 무료 (오픈소스) / 기업 라이선스 (월 \$500-5000) / 통합 컨설팅 (\$10-100k 프로젝트 단위)
- 유사 모델: Mapbox (지도), Auth0 (인증) 처럼 인프라성 SaaS

→ 이게 PRD §10 “후속 확장” 의 “방산/제조 클라이언트, SaaS 잠재 고객” 의 의미.

11. 더 깊이 공부하고 싶을 때

11-1. 책

책	주제	난이도
Anderson, "Aircraft Performance and Design"	항공 역학 입문	★★
Stevens, Lewis, "Aircraft Control and Simulation"	시뮬레이션 + 제어	★★★★
Cook, "Flight Dynamics Principles"	비행 동역학	★★★★
Ericson, "Real-Time Collision Detection"	충돌 검사 (게임 물리)	★★
Akenine-Möller et al., "Real-Time Rendering"	3D 렌더링 정석	★★★★

11-2. 무료 온라인

- NASA Glenn Research Center "Beginner's Guide to Aerodynamics": 무료, 기초 다지기
- MIT OpenCourseWare 16.07 (Dynamics): 6-DOF 수학적 기초
- PX4 Devguide: <https://docs.px4.io/> — 실 드론 SW 어떻게 만드나
- JSBSim Reference Manual: 정통 공력 모델 사양
- Three.js journey (Bruno Simon): 유료지만 3D 웹 그래픽 입문 최고

11-3. 코드로 배우기

- PX4 GitHub: 실 드론 자동조종 코드, BSD-3 → 수정 가능
- JSBSim GitHub: 정통 공력 모델
- FlightGear: 풀 시뮬레이터 오픈소스 코드
- Three.js examples: <https://threejs.org/examples/> — 우리가 안 다룬 기법 다수

11-4. 한국 커뮤니티

- KSAS (한국항공우주학회): 학술 모임, 연 2회 학술대회
- 항공우주산학연합의체: KAI 주도, 산학 협력
- 드론산업협회: 산업체 네트워킹

- GitHub의 한국 PX4 한글 커뮤니티: 활발하지 않지만 존재

11-5. 커리어 경로

이 분야에서 일하고 싶다면:

1. **게임 스튜디오 트랙:** Unity/Unreal + C++ → MS Asobo, 한국 게임사 (단, 비행 시뮬은 한국에 거의 없음)
2. **방산 트랙:** C++ + MATLAB/Simulink + 보안 클리어런스 → KAI, LIG, 한화
3. **UAM 스타트업 트랙:** Python + ROS + PX4 → 슈퍼널, Plana, 미국 Joby/Archer
4. **연구 트랙:** 박사 과정 (KAIST 항공우주, 서울대 기계항공) → 학계 + 정부 출연 연구소 (KARI, ADD)

연봉 기준 (한국, 2024): - KAI/LIG/한화 신입 연구직: 5500-7000만원 - UAM 스타트업: 6000-9000만원 (스톡옵션 포함) - 미국 Joby/Skydio: \$130-180k base + equity

12. 우리 프로젝트의 다음 마일스톤 (M10+) 추천

지금까지 우리는 M0 ~ M9를 마쳤다. 다음 마일스톤 후보들을 우선순위 별로 정리.

12-1. 학습 가치 우선

- **M10. JSBSim 통합** (학습: 정통 공력 모델). 우리 단순한 양력 공식을 진짜 NACA-기반 6-DOF 모델로 대체. WASM 빌드 + JS 인터페이스. **학습 효과 가장 큼.**
- **M11. ARINC 661 PFD**: 인증급 글래스 콕핏 디스플레이 → 항공 디스플레이 표준 학습

12-2. 데모/사업 가치 우선

- **M12. 한국 항공 (RKSI 인천) 정확 지형**: Mapbox 또는 SRTM → 실제 활주로/지형. 데모 가치 큼
- **M13. 다중 vehicle 클래스 정식**: 헬기, 전투기 추가. 비행 거동이 완전히 다르므로 학습 효과도 큼
- **M14. SaaS 상품화 prep**: 라이선스 모델, 가격 책정, 랜딩 페이지

12-3. 즐거움 우선

- **M15. 풀 사운드 디자인**: 절차적 사운드를 진짜 샘플로 교체 + 환경음 (활주로 ATC chatter, 무선 통신)
- **M16. 미션 에디터**: 사용자가 직접 시나리오 만들기. 공유 가능
- **M17. AI 적기/우군**: 단순한 dogfight 또는 형성비행 AI

추천: M10 (JSBSim 통합)

이유: 1. 비행 모델의 정확도가 한 단계 점프 (게임 → 인증급 근접) 2. 산업체 무료 자산 (JSBSim 모델 파일은 수십 종 공개)
3. 학습 효과 가장 큼 (분야의 “진짜 모델”이 어떻게 생겼는지 직접 봄) 4. AP 정확도 문제도 부수적으로 해결 (제대로 된 항 공기 거동)

언제든 다른 마일스톤 골라도 됨. 함께 가자.

마무리

이 문서를 끝까지 읽었으면 너는 이제 **flight-sim 분야의 95%를 알고 있다**. 나머지 5%는 실제 기체를 타보는 것 + 인증 절차의 디테일 정도이고, 그건 직업으로 들어가서 배워야 한다.

기억해야 할 핵심:

1. 비행 시물은 4개 시장 (게임 / 훈련 / 방산 / 연구) 이며 우리는 연구+엔터테인먼트 사이
2. 물리는 양력 + 항력 + 추력 + 중력, AoA에서 실속, 6-DOF 적분
3. 좌표계 부호 버그가 모든 시물의 가장 흔한 버그
4. MAVLink는 Dronecode 표준, 우리 브릿지가 그걸 구현
5. JSBSim, PX4, ROS, FlightGear가 OSS 4대 기둥
6. CAE, L3Harris, Lockheed가 글로벌 훈련/방산 시물 1티어
7. 한국은 KAI/LIG/한화 (방산) + 슈퍼널/Plana (UAM) + 두산 (드론)
8. DO-178C가 인증 SW 표준, FAA Level D가 시물 최고 등급
9. 우리 코드 5000줄은 100년의 표준 모자이크
10. 다음 큰 점프는 JSBSim 통합 (M10)

질문 / 다른 챕터 / 더 깊은 내용 원하면 알려줘. 함께 키워가자.